The whole-game differential — design, and the research it is taken from

Version 3.57.0 · Last updated 2026-08-06

ROADMAP #68. Written 2026-08-06 from Will's specification: *"we would want to play n games with like a thousand different teams to really test every single mechanic in the game"*, under his bar: *"i dont care if it takes a while, i just want it done correctly so we can trust it before using it to make all these decisions."*

---

1. Why the instruments we already have cannot answer the question

Every part is proven and nothing proves the whole.

instrument	reads	what it establishes	what it cannot see
mechanics census	235 probed, 234 live, **235 armed, 0 unarmed, 0 directCall**	each mechanic fires, and each probe was shown RED on a broken engine first	anything about **sequencing**
damage differential	149 / 150	damage matches Showdown on **staged pairs**	any error under its **12% tolerance**, uncounted
interaction matrix	1,624 / 1,643	pairs of mechanics compose	still staged, still pairwise
`data/game-diff.json`	**5 games**	five hand-written scenarios, 6 turns each	its `not_compared` list excludes hp amounts, accuracy misses, chance secondaries, crits, **any pair where one engine KOs and the other does not**, the `protect` volatile, and Showdown-only volatiles

Two of those deserve their own note.

The damage differential's tolerance is not absorbing roll variance. `showdownDamage()` is called at `roll=0` and `roll=15` against MEDICHAM's min and max, so both engines emit a pinned 16-roll range and the comparison is endpoint-to-endpoint. **A correct engine scores 0.000 on every row.** The 12% is free slack, and the artifact publishes only `worst`, so whether the 149 passing rows sat at 0.000 or at 0.119 is recorded by nothing.

The 31.1-point win-probability disagreement quoted in `docs/SUMMARY.md` predates WIRES 123–132. `engine/rerun_list.js` classes it unquotable. We do not currently know what that gap is.

---

2. The research this design is taken from

Each citation drives one decision below. They are not decoration.

2.1 Differential testing itself — McKeeman (1998)

Differential Testing for Software, Bill McKeeman, Digital Technical Journal. PDF

The founding statement: when two or more implementations of the same specification exist, feed both the same mechanically generated input and compare. **The oracle is free.** This is a materially stronger position than fuzzing, which can only see crashes and hangs — a differential harness sees *wrong answers*, which is the entire class of bug this project keeps finding.

McKeeman's other contribution is the **input tier**: he tested C compilers across seven tiers, from random ASCII up to model-conforming C programs. Deeper tiers find deeper bugs. Our tiers are already built and we are standing on the third of four: staged pair → staged interaction → **real game** → real game with real decisions.

Decision it drives: the harness compares two engines rather than asserting against expectations. We already own the authority (`engine/champions_sim.js`, the official simulator) and ADR-002 already says Showdown is it.

2.2 The input must make a difference *definitionally* a bug — Csmith (PLDI 2011)

Finding and Understanding Bugs in C Compilers, Yang, Chen, Eide, Regehr. preprint · ACM

325+ previously unknown bugs over three years. **Every compiler they tested both crashed and silently generated wrong code on valid input.** The methodological core is not the random generator — it is that Csmith generates programs *avoiding undefined and unspecified behaviour*, because where the standard permits two answers, a difference is not evidence of a bug and the oracle collapses.

Our undefined behaviour is **the dice.** Where either engine may legally roll, a divergence proves nothing. This project has already paid for that lesson once: CHANGELOG 3.45.0 records that the interaction matrix's *"two pinned dice were not the same die, so every sub-100-accuracy move had been MISSING in the reference engine while medicham2 hit"* — a whole class of false disagreement from one unpinned roll.

Decision it drives: two modes, and never one blurred mode. See §4.

2.3 Diversity comes from OMITTING features, not adding them — Swarm Testing (ISSTA 2012)

Swarm Testing, Groce, Zhang, Eide, Chen, Regehr. PDF · ACM

This is the paper that turns Will's *"a thousand different teams"* from a good instinct into a technique, and it says something counter-intuitive enough to be worth stating carefully.

Swarm testing generates a large population of configurations **each of which deliberately omits some features**, rather than one configuration that enables everything. In a week of testing it found **42% more distinct ways to crash a collection of C compilers than the heavily hand-tuned default configuration** of the same random tester.

The mechanism is the part that matters to us, and the paper gives it in two halves:

1. **Some features actively suppress the behaviour you want to reach.** Their example is a stack: including `pop` calls prevents the tester from ever driving the stack into its overflow-detection logic. **Our `pop` is Protect.** It is on almost every competitive set, it is the single most common click in the format, and a turn spent Protecting is a turn in which no damage, no secondary, no ability trigger and no field interaction occurs. A swarm that samples real teams uniformly will spend an enormous fraction of its turns testing nothing.
2. **Features compete for space.** Every slot spent on one mechanic is a slot not spent on another, so a uniform sample explores each mechanic shallowly. Omission frees the depth.

Decision it drives: team sampling is a *swarm over feature omission*, not a uniform draw from the meta. See §3.

2.4 A divergence is not actionable until it is minimal — Delta Debugging

Simplifying and Isolating Failure-Inducing Input, Zeller & Hildebrandt (the `ddmin` algorithm). retrospective · The Fuzzing Book's treatment `ddmin` systematically removes parts of a failing input and re-runs, isolating a **1-minimal** subset — one where removing any single remaining element makes the failure disappear. Worst case is $O(n^2)$; in practice far better, and later work (HDD, Perses) exploits input structure to do better still.

Why we need it: a divergence found in turn 5 of a 4-vs-4 game with eight Pokémon, thirty-two moves and a field state is a *report*, not a *bug*. Nobody can act on it. Reduced to "these two Pokémon, this one move, turn 1", it is a WIRE.

Decision it drives: the reducer is part of the harness, not a follow-up. See §5.

---

3. Team selection — a swarm, not a sample

Teams come from `data/games.ladder.jsonl`, 46,612 real games with real items, spreads and movesets. Real teams matter because a generated team is a team nobody would bring, and the mechanics that matter are the ones people actually click.

But drawing 1,000 teams uniformly from the meta is the *heavily hand-tuned default configuration* that Swarm Testing beats by 42%. Every top team shares the same features — Protect on three of four, Fake Out, the same two weather setters — so a uniform draw tests those deeply and everything else never.

So: **sample a swarm of configurations, each omitting a feature class**, and draw teams that satisfy each. Candidate omission axes, each of which unblocks something it currently suppresses:

omit	what it lets the run finally reach
Protect	turns that actually resolve — damage, secondaries, contact abilities, item triggers
priority moves	the ordinary speed-order path, and dynamic re-sorting (WIRE 118)
weather setters	terrain, and the no-weather damage path
Intimidate	attack stages at their declared values, and the crit interaction in §6
KO-capable attackers	long games, which is where residual damage, sleep counters and field expiry live
single-target moves	spread damage, Wide Guard, redirection

Report the swarm's composition in the artifact. A configuration that produced no games is a configuration that tested nothing, and must say so.

---

4. Two modes, never blurred

This is the Csmith lesson applied literally: a comparison is only evidence where a difference is *definitionally* a bug.

Mode A — PINNED. Tests mechanics and sequencing. Tolerance zero.

Every die fixed identically in both engines: no accuracy misses, no crits, no chance secondaries, damage roll pinned to one index on both sides. In this mode the two engines are **deterministic functions of the same input**, so *any* difference is a bug — no tolerance, no statistics, no sampling error. This is where the WIREs will come from.

The one subtlety, already paid for once: pinning must be the **same** die. Showdown's `randomChance()` bypasses a `battle.random` override entirely and crits need `willCrit`; `tests/test-engine-diff.js` records both traps in its header and cost `engine/validate_damage_sim.js` two debugging rounds. Reuse that code rather than re-deriving it.

Mode B — ROLLED. Tests rates. Compares distributions.

Everything Mode A pins is exactly what Mode A cannot test: **the crit rate is 1/24, accuracy is a distribution, secondaries are a chance.** Pinning them off proves nothing about them. So Mode B runs many seeds and compares *distributions*, not individual games — a chi-square or a confidence interval on "how often did a crit land", not "did this turn match".

The two modes answer different questions and neither substitutes for the other. Reporting them as one number would be the 12%-tolerance mistake again, one level up.

---

5. Naming what is different — the harness must localise, not just detect

Will, 2026-08-06: *"make sure the harness identifies what exactly is different between showdown and medicham so we can isolate what is wired incorrectly."* This is the requirement that separates a debugging instrument from a scoreboard, and it changes what gets compared.

Do not diff end-of-turn state. "Turn 4 HP differs by 12" is a symptom with a dozen possible causes — a damage multiplier, a missed residual, an item that should have triggered, a mis-ordered action — and it puts the reader back at the start.

Diff the EVENT STREAM, and use Showdown's own vocabulary as the common format. The official simulator already emits a structured, ordered protocol log:

```
|move|p1a: Incineroar|Fake Out|p2a: Garchomp
|-damage|p2a: Garchomp|154/175
|-ability|p1a: Incineroar|Intimidate|boost
|-unboost|p2a: Garchomp|atk|1
|-weather|Sandstorm|[upkeep]
|-damage|p1a: Incineroar|160/175|[from] Sandstorm
|-enditem|p2a: Garchomp|Focus Sash
```

That is not merely a log — it is a **step-level trace of every decision the authority made, already labelled with the mechanism that made it.** So: have MEDICHAM emit the same event shapes, align the two streams, and the **first differing event names the wired-wrong thing directly.** A missing `|-unboost|` is Intimidate. A `|-damage|` with the wrong number after identical preceding events is the damage math. An out-of-order `|move|` pair is turn order. An absent `|-enditem|` is the Sash.

This is the same reasoning that makes differential testing work at all (§2.1), applied one level down: the finer the granularity at which two implementations are compared, the more directly the difference points at its cause. McKeeman compared program *outputs*; comparing *traces* is strictly more informative when both sides can be made to emit one.

Each divergence is then classified and counted by class, not by instance. The report should read

```
turn order          12 games   3 distinct causes
residual damage      8 games   1 cause
item trigger         5 games   2 causes
stat stage           4 games   1 cause
```

because the point is to fix a class of wiring, and 12 instances of one turn-order bug is one WIRE, not twelve findings. This is how the arming pass worked: WIRE 129 was one root cause under six dead arms.

Then, and only then, the mechanical reduction:

1. **First divergence per game only.** Everything after the first is downstream of it, and counting consequences as separate findings inflates the number and hides the cause.
2. **The divergence, reduced.** Run `ddmin` over the game: drop turns, drop Pokémon, drop moves, drop items, keeping only what still diverges. Ship the 1-minimal repro, not the game — and because the trace already names the event, the reduction has a target to preserve rather than a vague "still fails" predicate, which is exactly the structure-aware reduction later delta-debugging work (HDD, Perses) exploits.
3. **The repro must be emitted as a runnable probe**, in the shape `tests/probe_red_demo.js` already takes, so a divergence converts directly into an armed census probe. The harness's output is then not a report anybody has to transcribe — it is the next test.
4. **The run's own mechanic coverage.** Which of the 235 census mechanics were actually exercised. **A run that never triggered Illusion has not tested Illusion** and must say so rather than reading as a clean sweep. This is the project's own standing rule — *a capability that cannot prove it ran is assumed broken* — pointed at the harness itself, and it is what makes Will's "every single mechanic" a measurement rather than an aspiration.
5. **The swarm composition**, per §3.
6. **A frozen engine release id.** It is a measurement; `engine/engine_release.js` and the photograph rule apply.

---

6. A worked example of what Mode A should catch on day one

Three parts of a critical hit are declared unmodelled in `engine/medicham2-browser.js`, confirmed against Showdown's `sim/battle-actions.ts:1683-1691`:

```
if (isCrit) { ignoreNegativeOffensive = true; ignorePositiveDefensive = true; }
const ignoreOffensive = !!(move.ignoreOffensive || (ignoreNegativeOffensive && atkBoosts < 0));
const ignoreDefensive = !!(move.ignoreDefensive || (ignorePositiveDefensive && defBoosts > 0));
```

A crit ignores the **attacker's negative** offensive stages, the **defender's positive** defensive stages, and screens. It does **not** ignore burn — those rules operate on boost *stages*, and burn is an `onModifyAtk` multiplier. Gen 2 ignored burn on a crit and Gen 3 onward does not, which is why that half is the one people misremember.

The first of the three is the expensive one here: **Intimidate is everywhere in this format**, so an Intimidated attacker landing a crit should hit at full Attack, and MEDICHAM prices it at −1. A `willCrit` move (Flower Trick, Storm Throw, Frost Breath) thrown by an Intimidated attacker in Mode A is a two-line scenario that separates the engines exactly — and no staged pair test we own is shaped to notice, because the census asks *does the mechanic fire*, not *is the number right afterwards*.

---

7. Cost, and what is honestly unknown

Building it is roughly a day: both engines already play games, `tests/test-engine-diff.js` already solves the die-pinning traps, and the store already holds the teams. Running it is cheap — MEDICHAM measures ~1,600–2,000 battles/sec, so 1,000 games is under a minute and the official engine is the slower side.

The fix cycle afterwards is not estimable, and saying otherwise would be a number to retract later. Nobody knows what it finds, because it is the first instrument that could see a sequencing bug at all. The base rate in this repository is bad: every deep instrument built so far found real bugs immediately — the arming pass found seven, the interaction matrix found twelve, the accuracy probe found that all four doors were dead. Csmith found 325 bugs in compilers that had been in production for decades and were tested far harder than this. Expect several WIREs.

Done looks like: divergences per 1,000 games is small, every survivor has a written reason, and the run's own coverage is high enough that "we tested it" is a measurement rather than a claim. Until then MEDICHAM's output is not trusted for decisions — which is exactly the bar Will set.

Sources

- McKeeman, *Differential Testing for Software* — https://www.cs.tufts.edu/comp/150FP/archive/bill-mckeeman/DifferentailTesting.pdf
- Yang, Chen, Eide, Regehr, *Finding and Understanding Bugs in C Compilers* (PLDI 2011) — https://users.cs.utah.edu/~regehr/papers/pldi11-preprint.pdf
- Groce, Zhang, Eide, Chen, Regehr, *Swarm Testing* (ISSTA 2012) — https://agroce.github.io/issta12.pdf
- Zeller & Hildebrandt, *Simplifying and Isolating Failure-Inducing Input* (delta debugging / `ddmin`) — https://www.computer.org/csdl/journal/ts/2025/03/10859156/23X97jMgYjm
- Groce et al., *Randomized Differential Testing as a Prelude to Formal Verification* (ICSE 2007) — https://agroce.github.io/icse07.pdf
- Reducing failure-inducing inputs, The Fuzzing Book — https://www.fuzzingbook.org/html/Reducer.html